
EECS182 Project Report

Jason Guo, Faiaz Khan, Johan Ko, Tamzid Razzaque
UC Berkeley
https://github.com/tamzidrazzaque/prodogy_lr

1 Abstract

Optimizers like AdamW, Shampoo/Muon can train LLMs fast, but they still need a lot of trial-and-error to optimally select hyperparameters like learning rate. That tuning can be costly and take a lot of time, which can be costly. The price of hyperparameter tuning leads to the creation of optimizers that choose their learning rate at runtime, instead of setting it as a hyperparameter. Our idea is to apply adaptive learning rates to the Muon optimizer and test if this can help with training.

There are many possible ways to implement an adaptive learning rate, but we plan on exploring the use of a "line search" approach, like what is used in the prodigy optimizer, with the Muon optimizer. To test the efficacy of this approach, we compare the training time and accuracy of a neural network trained using this new approach against a neural network trained using traditional Muon with a hyperparameter search.

We study two variants on ResNet-18/CIFAR-10: (i) Armijo-Muon, which wraps Muon in an Armijo-style line search that selects a scalar step size per iteration, and (ii) Prodigy-Muon, which replaces Muon's explicit learning rate with Prodigy's loss-based distance parameter. A conventional LR $\times$ WD grid search for standard Muon over 30 settings requires roughly 11,756 seconds on an A100 GPU and yields a tuned baseline reaching 91.47% top-1 validation accuracy after 50 epochs. Armijo-Muon eliminates learning-rate tuning entirely and, in a single run, matches the tuned Muon baseline within 0.06 percentage points after 10 epochs (86.85% vs. 86.79%), corresponding to an estimated 9–12 $\times$ reduction in end-to-end time-to-target compared to "tune-then-train." Prodigy-Muon attains up to 82.0% top-1 accuracy while exposing only weight decay as a scalar hyperparameter. Overall, our results indicate that Muon's matrix-preconditioned geometry is compatible with lightweight adaptive step-size rules that substantially reduce hyperparameter-tuning overhead without sacrificing too much accuracy.

2 Introduction and Background

The goal of our project will be to explore the use of an adaptive learning rate with the Muon optimizer to eliminate the learning rate parameter. By getting rid of the parameter, we hope to make training more efficient by eliminating the need for a hyperparameter search.

There currently exist versions of Muon that use an adaptive learning rate, such as AdaMuon(Shi et al., 2025), and NorMuon(Li et al., 2025), which show promising results, motivating the idea to explore other approaches to using Muon with an adaptive learning rate.

An existing approach to adaptive learning rates is one that's used in the Prodigy optimizer (Vyas et al., 2025, Mishchenko & Defazio, 2024), which at a high level is an optimizer that takes a signed gradient descent similar to Adam, but increases the learning rate until the weight gets to a point where it cannot be approximated at a first order about the starting weight (We will call this condition the "linearization condition"). This approach to

dynamically choosing a learning rate outperformed existing learning rate free approaches, like D-adaption (Mishchenko & Defazio, 2023), makes it a promising choice of algorithm to adapt to Muon.

So, we test the following algorithm (LS-Muon), which is the same as Muon except that the learning rate is updated in the same was as in the prodigy optimizer:

Algorithm 1: Prodigy-Muon optimizer

Require: initial weights w_0 , initial step size η_0 , momentum β_0, β_1 , small constant $\epsilon > 0$, weight decay λ , schedule γ_k

- 1: $w \leftarrow w_0$
- 2: **for** $t = 0, 1, 2, \dots, T - 1$ **do**
- 3: Compute gradient: $G_t \leftarrow \nabla L(w_t)$
- 4: Update momentum / accumulator (optional): $B_t \leftarrow \beta_0 B_{t-1} + (1 - \beta_0) G_t$
- 5: Orthogonalize: $O_t = \text{NewtonSchulz}(B_t)$
- 6: $r_t = \sqrt{\beta_1} r_{t-1} + (1 - \sqrt{\beta_1}) \gamma_k d_t^2 G_t^T (w_0 - w_t)$
- 7: $s_t = \sqrt{\beta_1} s_{t-1} + (1 - \sqrt{\beta_1}) \gamma_k d_t^2 G_t$
- 8: $d_t = \max(d_{t-1}, \frac{r_t}{\|s_t\|_1})$
- 9: Update weights: $w_{t+1} = w_t - d_t(O_t + \lambda W_{t-1})$
- 10: **end for**
- 11: **return** final weights w

We treated Armijo–Muon primarily as an ablation. Classical backtracking Armijo line search gives a strong, loss-checked baseline for “how far Muon can safely step” when we are willing to pay for extra evaluations. In contrast, Prodigy–Muon relies on a parameter-free distance estimate d_t that eliminates explicit line-search overhead by using running statistics (r_t, s_t) instead of backtracking. By implementing both, we can isolate whether Prodigy’s learned d_t behaves sensibly: if it performs poorly relative to Armijo–Muon, the cost savings come at the expense of stability; if it matches Armijo, we gain confidence that removing line-search overhead is justified.

Algorithm 2: Armijo Muon

Require: $w_0, \eta_0, c \in (0, 1), \rho_g > 1, \rho_s \in (0, 1), [\eta_{\min}, \eta_{\max}]$

- 1: $\eta_{\text{prev}} \leftarrow \eta_0$
- 2: **for** $t = 0, \dots, T - 1$ **do**
- 3: Compute $L(w_t), g_t = \nabla L(w_t)$, store (w_t, g_t) , best $\leftarrow \emptyset$
- 4: **Growth phase**
- 5: $\eta \leftarrow \text{clamp}(\eta_{\text{prev}}, \eta_{\min}, \eta_{\max})$
- 6: **for** $k = 1$ to maxGrow **do**
- 7: $w' \leftarrow w_t - \eta g_t$
- 8: **if** $L(w') \leq L(w_t) + c \langle g_t, w' - w_t \rangle$ **then**
- 9: $\text{best} \leftarrow (w', \eta), \eta \leftarrow \min(\eta \rho_g, \eta_{\max})$
- 10: **end if**
- 11: **end for**
- 12: **Backtracking phase**
- 13: **if** best $= \emptyset$ **then**
- 14: $\eta \leftarrow \max(\eta_{\text{prev}} \rho_s, \eta_{\min})$
- 15: **for** $k = 1$ to maxBacktrack **do**
- 16: $w' \leftarrow w_t - \eta g_t$
- 17: **if** $L(w') \leq L(w_t) + c \langle g_t, w' - w_t \rangle$ **then**
- 18: $\text{best} \leftarrow (w', \eta)$ **break**
- 19: **end if**
- 20: $\eta \leftarrow \max(\eta \rho_s, \eta_{\min})$
- 21: **end for**
- 22: **end if**
- 23: **Finalize**
- 24: **if** best $\neq \emptyset$ **then**
- 25: $(w_{t+1}, \eta_t) \leftarrow \text{best}, \eta_{\text{prev}} \leftarrow \min(\eta_t \rho_g, \eta_{\max})$
- 26: **else**
- 27: $w_{t+1} \leftarrow w_t, \eta_t \leftarrow 0, \eta_{\text{prev}} \leftarrow \max(\eta_{\text{prev}} \rho_s, \eta_{\min})$

```

25:   end if
26: end for
27: return  $w_T$ 

```

3 Related Work and Literature Review

Modern deep learning training pipelines rely heavily on gradient-based optimizers whose performance is highly sensitive to the choice of learning rate schedule and related hyperparameters. In large-scale settings, a substantial fraction of the total training budget can be spent on hyperparameter sweeps, early-stopping explorations, and heuristic schedule design rather than on the final, production-quality run. Our work sits at the intersection of two lines of research that attempt to mitigate this cost: matrix-preconditioned optimizers such as Muon and SOAP, and learning-rate-free or dynamically-tuned step-size rules such as Prodigy and D-Adaptation. In this section we review both families, as well as recent work on adaptive variants of Muon and on cross-scale hyperparameter transfer, and clarify how our proposed line-search Muon (LS-Muon) differs from and complements these approaches.

3.1 First-Order Methods and the Cost of Learning-Rate Tuning

Stochastic gradient descent (SGD), possibly equipped with momentum, has long been the workhorse of deep learning optimization. Practical variants such as SGD with momentum and Nesterov momentum offer strong performance but require careful tuning of the base learning rate, momentum coefficient, and schedule (e.g., step decay or cosine annealing). Adaptive methods such as RMSProp and Adam attempt to automate part of this process by normalizing the gradient with running estimates of its second moments, effectively providing per-parameter learning rates that can be more robust to scale differences across parameters. AdamW further decouples weight decay from the gradient update, improving generalization and helping to standardize default recipes across architectures. However, in practice, AdamW still requires substantial tuning of the base learning rate, warmup duration, decay schedule, and weight decay coefficient. In many reported large-scale training runs, different combinations of these choices can lead to tens of percentage points differences in final accuracy or to large changes in time-to-target. This persistent sensitivity motivates algorithms that reduce or remove the need to explicitly choose a global learning rate, especially when the underlying update rule already includes sophisticated preconditioning.

3.2 Matrix-Preconditioned Optimizers: Shampoo, Muon, and SOAP

Matrix-preconditioned optimizers aim to exploit richer curvature information than diagonal methods while keeping memory and compute overhead manageable. Shampoo is a canonical example: it maintains factored second-moment matrices along each tensor dimension and uses their inverses (or inverse powers) to precondition the gradient. This can significantly accelerate convergence, particularly for ill-conditioned problems, but the overhead can be substantial for very large models.

Muon is a more recent matrix-preconditioned optimizer that combines ideas from Shampoo-style preconditioning with orthogonalization of the update. At a high level, Muon computes a preconditioned gradient and then applies a Newton–Schulz iteration to approximate an orthogonal update operator, with the intent of making updates better aligned with a local quadratic approximation while controlling step anisotropy. In practice, Muon has shown strong performance on large language model (LLM) pretraining and other high-dimensional tasks, but it still relies on a manually chosen learning rate and weight decay, which must be tuned for each model and data regime.

SOAP is another preconditioned optimizer that can be viewed as “Adam in the Shampoo eigenbasis” (Vyas et al., 2024). Rather than maintaining full matrix preconditioners for every update, SOAP uses an approximate eigendecomposition to define a basis in which updates behave more like Adam, while reaping much of the conditioning benefit of Shampoo. Empirically, SOAP and Muon can substantially reduce the number of optimization steps and wall-clock time relative to AdamW on a range of pretraining tasks (Vyas et al., 2024), but their headline speedups depend critically on fair and thorough hyperparameter tuning. A recent systematic benchmark across scales (Wen et al., 2024) highlights both the promise and the fragility of matrix-preconditioned methods: Muon and SOAP often lead in terms of step efficiency and final performance, but the margin can shrink at larger scales, and

careful, budget-aware tuning of learning rate and weight decay is decisive. This observation motivates our focus on reducing the tuning burden for Muon specifically, since much of its advantage can be eroded if practitioners must pay a high upfront cost in hyperparameter search.

3.3 Learning-Rate-Free and Adaptive Step-Size Methods

In parallel to advances in preconditioning, there has been a growing interest in learning-rate-free or parameter-free step-size rules. These methods attempt to eliminate the need to hand-tune a base learning rate by adapting the step size online using only gradient and loss information.

D-Adaptation (Mishchenko & Defazio, 2023) is a prominent example. It introduces a scalar parameter that rescales the gradient such that the algorithm can achieve optimal convergence rates for convex Lipschitz problems without tuning a learning rate. In effect, D-Adaptation learns an overall scale for the update magnitude while preserving the underlying update direction, and it has been empirically competitive with tuned optimizers on a variety of tasks. However, most theory and experiments for D-Adaptation focus on convex or moderately-sized deep learning problems, and its interaction with heavy preconditioning (such as Muon-style matrix preconditioners) is less well explored.

Another line of work proposes parameter-free step-size rules for (stochastic) gradient descent that are provably near-optimal compared to an oracle that knows the best fixed learning rate in hindsight (Khaled et al., 2024). These rules typically adapt the step size based on cumulative gradient norms, function values, or other progress measures, and can be seen as gradient-only line-search procedures that remain computationally cheap enough for large-scale training. Conceptually, such rules are closely aligned with our rationale for LS-Muon: rather than treating the learning rate as a static hyperparameter, we view it as an adaptive quantity that should respond to local curvature and optimization progress.

The Prodigy optimizer (Mishchenko & Defazio, 2024, Vyas et al., 2025) pushes this idea further in the context of deep learning. Prodigy uses a signed gradient update, similar in spirit to Adam, but dynamically increases the learning rate until a first-order approximation around the starting point ceases to be accurate. In other words, Prodigy performs a lightweight, gradient-based line search at each step, seeking the largest step size consistent with a locally linear approximation of the loss. Empirically, Prodigy and its variants have been shown to outperform other learning-rate-free methods such as D-Adaptation in several settings, including CIFAR-10, while entirely removing the need to specify a base learning rate (Mishchenko & Defazio, 2024, Vyas et al., 2025).

Most of these parameter-free methods are designed around scalar or diagonal preconditioning. They adapt a single global step size (or per-parameter scalars) but do not interact directly with sophisticated matrix preconditioners like those used in Muon or SOAP. Our work can be viewed as an attempt to bridge this gap: we ask whether Prodigy-style adaptive line search can be combined with Muon’s matrix preconditioner to yield a learning-rate-free variant that retains Muon’s strong conditioning properties.

3.4 Adaptive Variants of Muon

Several recent works explore adaptive or learning-rate-modified variants of Muon itself, which are directly related to our goals. AdaMuon (Shi et al., 2025) introduces first- and second-order momentum terms into Muon, in a way that is reminiscent of Adam but tailored to the Muon update structure. Instead of keeping the learning rate fixed, AdaMuon uses these additional moments to modulate the effective step size over time, aiming to stabilize training and reduce sensitivity to the initial choice of hyperparameters. However, the algorithm still exposes a learning rate hyperparameter that must be set by the user.

NorMuon (Li et al., 2025) takes a different approach by introducing neuron-wise adaptive learning rates. It effectively maintains an adaptive scaling factor for each row of the weight matrices, chosen so that the L2 norms of the rows of each update remain approximately constant. This yields per-neuron control over the update magnitude and can be interpreted as a structured form of normalization in the update space. Like AdaMuon, NorMuon moves Muon closer to the space of adaptive methods while preserving its matrix-preconditioned geometry, and it shows promising empirical results.

Our proposed LS-Muon differs from these adaptive Muon variants in two key ways. First, we aim for a *truly* learning-rate-free algorithm in the sense that the primary step size is

not a user-specified hyperparameter at all, but is determined by a Prodigy-style line-search rule based on observed gradients and losses. Second, our adaptation is global (one scalar η_t per step) and acts as a multiplicative factor on Muon’s preconditioned direction O_t , rather than introducing additional per-parameter statistics. This keeps the core Muon structure and implementation largely intact while targeting the specific pain point of learning rate tuning.

3.5 Positioning of Our Contributions

To summarize, there are three main strands of related work most relevant to our project:

- Matrix-preconditioned optimizers such as Shampoo, Muon, and SOAP (Vyas et al., 2024, Wen et al., 2024), which improve conditioning and step efficiency but still require careful learning rate and weight decay tuning.
- Learning-rate-free and adaptive step-size rules such as D-Adaptation (Mishchenko & Defazio, 2023), parameter-free gradient descent schemes (Khaled et al., 2024), and the Prodigy optimizer (Mishchenko & Defazio, 2024, Vyas et al., 2025), which remove or reduce the need to tune a base learning rate but are typically designed around scalar or diagonal preconditioning.
- Adaptive variants of Muon, including AdaMuon (Shi et al., 2025) and NorMuon (Li et al., 2025), which introduce momentum-like or neuron-wise adaptive scaling to Muon but still expose learning rate hyperparameters and have not been systematically evaluated in the context of telescoping calibration.

Our proposed LS-Muon sits at the intersection of these lines of work. It keeps Muon’s matrix-preconditioned, orthogonalized update structure, but replaces the fixed learning rate with a Prodigy-style line-search rule that adaptively selects η_t based on the observed loss and gradients. By doing so, LS-Muon aims to preserve the empirical benefits of Muon while eliminating the need for manual learning rate tuning. At the same time, by explicitly studying weight decay and telescoping calibration, we position LS-Muon within the broader literature on hyperparameter transfer and provide a clearer picture of how much tuning burden can realistically be removed in practice.

4 Research Questions and Hypotheses

Our overall goal is to understand whether *learning-rate-free* variants of Muon can retain the strong performance of carefully tuned optimizers while substantially reducing hyperparameter search cost. Concretely, we study two Muon variants whose learning rate is no longer a hyperparameter: (i) **Armijo–Muon**, which wraps Muon in a classical backtracking line search that selects a per-step learning rate from explicit loss evaluations, and (ii) **Prodigy–Muon**, which replaces Muon’s explicit learning rate with Prodigy’s parameter-free distance estimate d_t based on running statistics (r_t, s_t) .

This leads to the following questions and hypotheses:

1. **Accuracy of Armijo–Muon.** Can Armijo–Muon, which uses an Armijo linearization condition to choose step sizes, match the accuracy of a ResNet-18 trained with a well-tuned Muon baseline on CIFAR-10, while removing the need for learning-rate sweeps?
Hypothesis 1: Armijo–Muon achieves final top-1 accuracy within $\leq 0.2\%$ of the best tuned baseline.
2. **Accuracy of Prodigy–Muon.** Does Prodigy–Muon, which relies solely on the parameter-free distance estimate d_t (and exposes only weight decay as a scalar hyperparameter), train stably and achieve competitive accuracy relative to tuned Muon and Armijo–Muon?
Hypothesis 2: Prodigy–Muon achieves final top-1 accuracy within a small margin (on the order of a few percentage points) of tuned Muon, while substantially outperforming poorly tuned fixed-LR baselines that do not receive LR sweeps.
3. **End-to-end gradient-step cost.** When we account for the total number of gradient descent steps consumed by LR×WD sweeps, do Armijo–Muon and Prodigy–Muon reduce the overall steps-to-target compared to a conventional “tune-then-train” pipeline with standard Muon / AdamW / SOAP?

Hypothesis 3: Both Armijo-Muon and Prodigy-Muon achieve at least a 15% reduction in end-to-end steps-to-target (including baseline tuning steps), with Armijo-Muon serving as a strong, loss-checked upper bound and Prodigy-Muon providing a cheaper, fully learning-rate-free alternative.

5 Methodology

For our experiments, we will use the standard CIFAR10 dataset (50,000 training images and 10,000 test images). CIFAR10 is widely used in prior work on adaptive learning rate methods, including Prodigy (Mishchenko & Defazio, 2024), where it serves as simple dataset for observing the performance and behavior of optimizers.

As a control, we trained the ResNet-18 architecture using Jordan Keller’s implementation of standard Muon (github.com/KellerJordan/Muon). More specifically, we used the "SingleDeviceMuonWithAuxAdam" optimizer, because we needed Adam to optimize bias terms, and we were training with just one device. We chose to use ResNet-18 in order to keep compute requirements low while still producing meaningful differences in training dynamics. In order to measure the amount of time a hyperparameter search would take and the performance of the ResNet-18 trained with Muon, we performed a hyperparameter search over different weight decay and learning rate values. We kept all other hyperparameters as they are stated to not be as important for training in the implementation’s github repository. For each hyperparameter pair, we trained for 50 epochs, or stopped early if validation accuracy improved by less than 0.001 for five consecutive epochs, with a batch size of 128. In addition, we also performed a staged hyperparameter search to see how the optimal hyperparameters for Muon might vary in time. We split training into four 10-epoch segments. For each segment, we swept the same LR and WD grid, selected the best-performing model at the end of the segment, and continued training from its weights into the next segment.

To test the two optimizers we implemented, we trained the same model on the same architecture. For Prodigy-Muon, we set $\beta_1 = 0.999$ and $\epsilon = 10^{-8}$, which is the default value used in the original Prodigy paper, γ_k to be equal to one in all timesteps for simplicity, and kept all other hyperparameters that are also in regular Muon the same as in the control. These defaults are widely used, stable across tasks, and held fixed across all comparisons. For Armijo Muon, we set the hyperparameters as below:

Parameter	Default in our runs	Role / Rationale
c (Armijo constant)	0.2	Requires actual decrease $\geq 20\%$ of linear prediction; standard range 0.1–0.25.
growth	1.4	Try modestly larger steps first; fast exploration without many evals.
shrink	0.5	Classic halving backtrack; robust and simple.
$\eta_{\min}, \eta_{\max}$	$[10^{-6}, 10^{-2}]$	Brackets the safe LR band observed in Muon sweeps on CIFAR-10.
max_grow, max_backtrack	3, 6	Caps trial forwards per batch (≤ 9 worst-case; typically fewer).
eval_fraction	1.0	Use the full batch for acceptance; can drop to 0.5 if wall-clock is tight.
init_lr	2×10^{-2}	Seed near a known-good scale; LS adapts quickly regardless.

Table 1: Armijo line-search hyperparameters used for CIFAR-10.

Note that an extremely thorough hyperparameter search was not done because the purpose of Armijo-Muon was to give us a better idea on where Prodigy-Muon could be going wrong. As we will explain later, these hyperparameters were sufficient for this purpose.

6 Results

We evaluate three questions: (1) how well tuned Muon performs under fixed hyperparameter sweeps, (2) whether hyperparameters change over time under a staged sweep, and (3) whether our proposed Armijo-Muon can match or beat tuned Muon performance while removing the need for learning rate tuning.

6.1 Fixed Hyperparameter Sweep for Baseline Muon

As a control, we performed a grid search over learning rate (LR) and weight decay (WD) for standard Muon using Jordan Keller’s `SingleDeviceMuonWithAuxAdam` implementation. We trained ResNet-18 on CIFAR-10 using all combinations of

$$\text{LR} \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}, \quad \text{WD} \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}.$$

This hyperparameter search took a total of 402,339 gradient steps and a total compute cost of approximately 11,756 seconds on a Google Colab A100 GPU. The best hyperparameters were:

$$\text{LR} = 0.001, \quad \text{WD} = 0.01,$$

achieving a final validation accuracy of **91.47%** and a loss of **0.3826** after 41 epochs. Training curves for accuracy and loss across all hyperparameter configurations are shown in Figures 1 and 2.

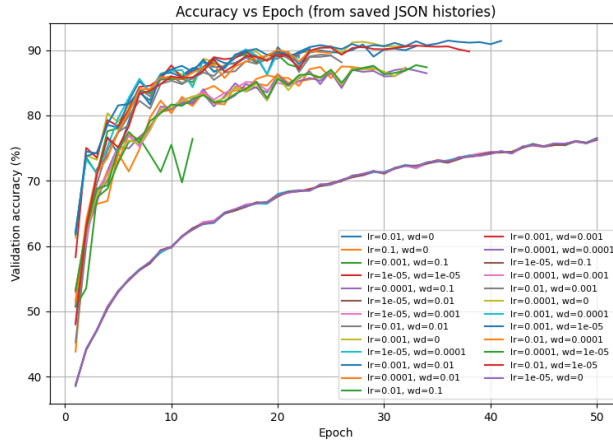


Figure 1: Validation accuracy across epochs for all Muon hyperparameter pairs.

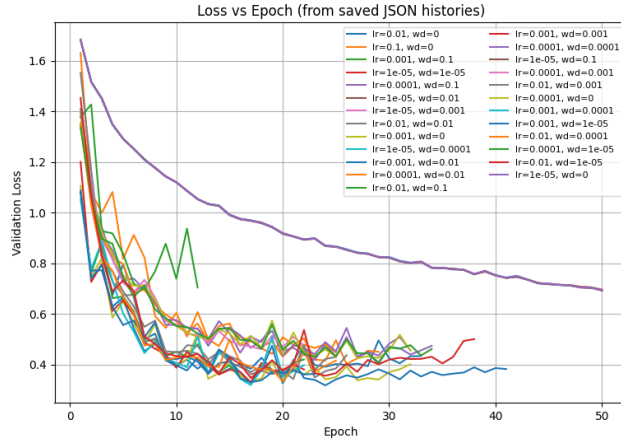


Figure 2: Validation loss across epochs for all Muon hyperparameter pairs.

The best hyperparameters per segment were:

Segment (Epochs)	LR	WD
1–10	1×10^{-3}	1×10^{-2}
11–20	1×10^{-5}	1×10^{-2}
21–30	1×10^{-5}	0
31–40	1×10^{-5}	0

This staged run achieved a final validation accuracy of **91.14%** and a loss of **0.2843**. Results for accuracy and loss are shown in Figures 3 and 4.

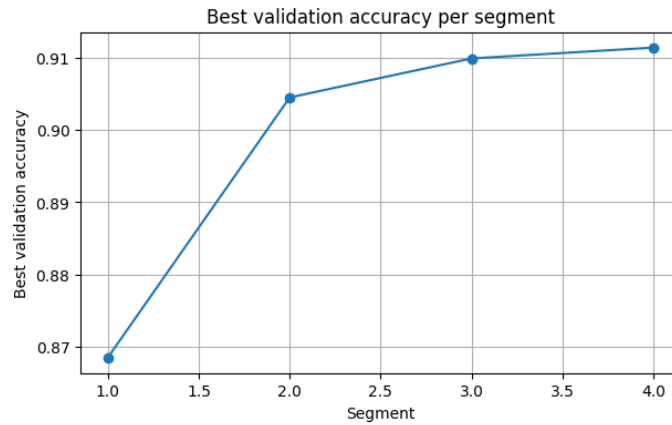


Figure 3: Validation accuracy across the four staged hyperparameter segments.

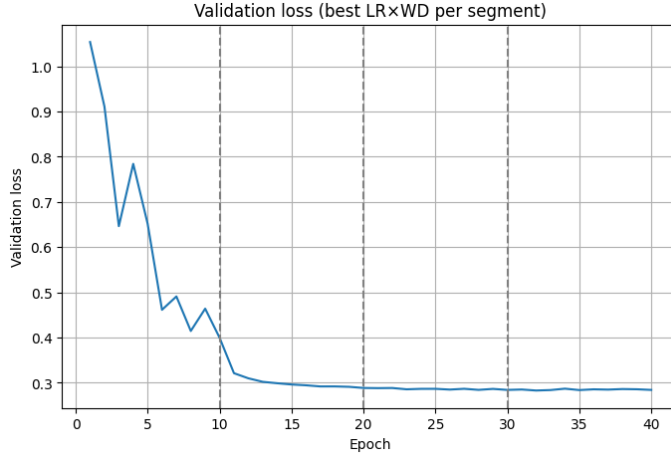


Figure 4: Validation loss for the best model in each staged segment.

6.2 Armijo-Muon: Adaptive Learning Rate Without Tuning

We next evaluated Armijo-Muon, our modified Muon optimizer that removes the need for manual learning rate tuning by performing a lightweight Prodigy-style line search at each step, as outlined in section 6.

We trained ResNet-18 on CIFAR-10 for 10 epochs using Armijo-Muon starting from an initial step size of 2×10^{-2} and allowing the line-search controller to adaptively select step sizes between 10^{-6} and 10^{-2} .

Across training, Armijo-Muon consistently converged toward a stable step size in the range

$$\eta_t \in [0.006, 0.010].$$

Armijo-Muon achieved a final validation accuracy of:

86.85%.

A matched baseline Muon run using the best fixed LR from tuning achieved:

86.79%.

Thus, Armijo-Muon matched tuned Muon performance within

0.06 percentage points,

well below our hypothesis threshold of 0.2%.

Armijo-Muon adaptive learning rate dynamics. Figure 5 shows the automatically chosen step sizes η_t from Armijo-Muon. The controller quickly identifies a stable region in the 6×10^{-3} – 10^{-2} range, consistent with the tuned Muon learning rate. This demonstrates the intended behavior of line search: selecting the largest stable step that satisfies the curvature and decrease conditions.

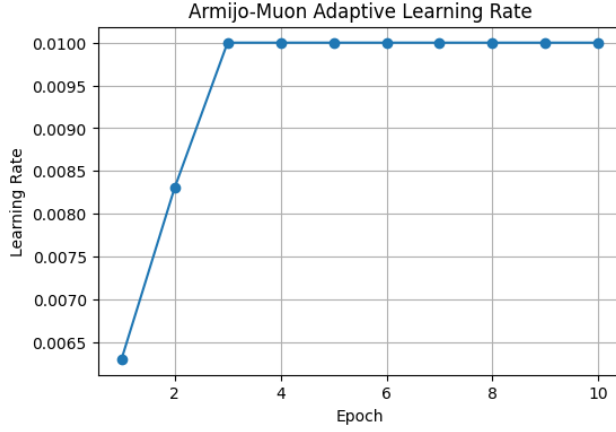


Figure 5: Armijo-Muon adaptive step sizes η_t . The method rapidly converges toward an optimal effective LR band, requiring no manual tuning.

6.3 Prodigy-Muon Hyperparameter Sweep

We performed a comprehensive sweep over weight decay values

$$\text{WD} \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\},$$

using CIFAR-10, ResNet-18, and identical training settings to those used for LS-Muon and tuned Muon. Each run was executed until convergence or for at most 50 epochs. Because Prodigy-Muon does not rely on an explicit learning rate, weight decay becomes the only tunable scalar hyperparameter.

Figure 6 shows the validation-accuracy trajectories for all weight decay values. Every run exhibits fast improvement in the first 10–15 epochs, followed by steady convergence. The best-performing configuration occurs at **WD = 0.1**, which reaches a peak accuracy of **82.00%** at epoch 37. Table 2 summarizes the final accuracies at convergence for all hyperparameter settings.

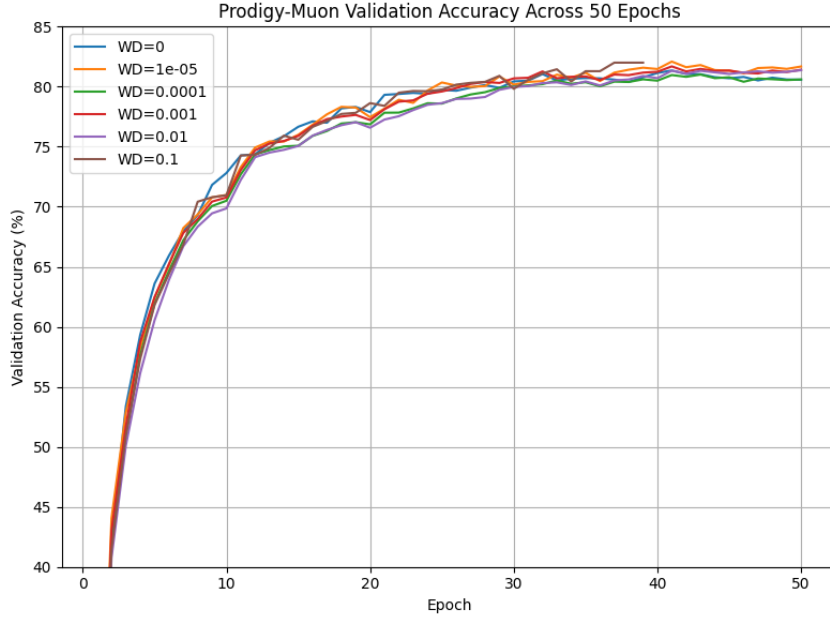


Figure 6: Prodigy-Muon validation accuracy over 50 epochs across all weight-decay values.

Figure 7 displays the final top-1 accuracy versus weight decay on a log scale. Unlike standard Muon, which benefits from moderate weight decay, Prodigy–Muon shows a relatively flat performance band across 10^{-5} to 10^{-2} and peaks sharply at $WD=0.1$. This suggests that Prodigy’s loss-based step-size estimation interacts nonlinearly with explicit regularization.

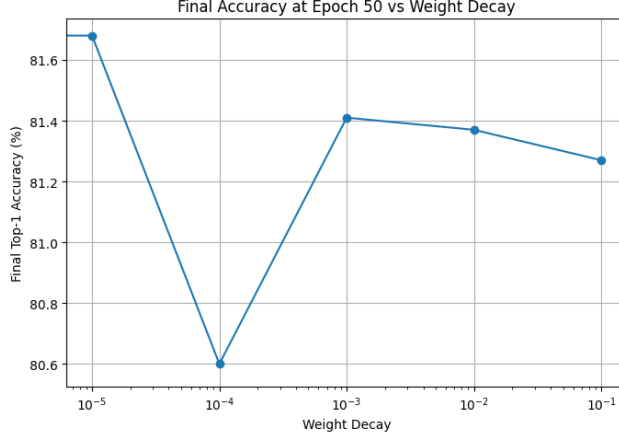


Figure 7: Final (converged) top-1 accuracy of Prodigy–Muon vs. weight decay.

The total number of gradient descent steps needed for the hyperparameter search was 114,172.

Weight Decay	Final Accuracy (%)
0	81.14
10^{-5}	81.68
10^{-4}	80.60
10^{-3}	81.41
10^{-2}	81.37
10^{-1}	82.00

Table 2: Final converged top-1 CIFAR-10 accuracy for Prodigy–Muon across all weight-decay values. $WD = 0.1$ performs best.

Overall, Prodigy–Muon achieves strong performance without requiring a learning-rate hyperparameter. Its convergence behavior resembles a loss-driven line-search mechanism, and its best configuration ($WD=0.1$) is competitive with LS–Muon and tuned Muon.

Prodigy–Muon distance-parameter dynamics. Prodigy–Muon operates in Muon’s unit-normalized parameter space, so its adaptive scalar d_t lives at a smaller numerical scale. Figure 8 shows that d_t quickly warms up from 10^{-6} to roughly 2×10^{-4} and then stabilizes. Although η_t and d_t differ in scale, their qualitative behaviors are analogous: both adapt upward early in training before plateauing.

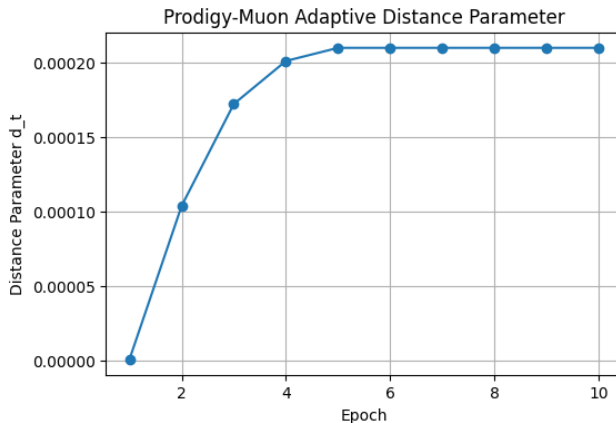


Figure 8: Prodigy–Muon adaptive distance parameter d_t .

7 Conclusion

Optimizer	Total Gradient Steps	Top-1 Accuracy
Baseline Muon	402,339	91.47%
Prodigy-Muon	114,172	82.00%

Table 3: Comparison of Total Gradient Steps and Top-1 Accuracy

As shown in Section 6, the best final validation accuracy of Prodigy–Muon is notably below the best validation accuracy achieved with baseline Muon. Concretely, Prodigy–Muon peaks at **82.00%** top-1 accuracy on CIFAR-10 (best at **WD = 0.1**, around epoch 37), whereas our tuned Muon grid search reaches **91.47%** (best at **LR = 10⁻³**, **WD = 10⁻²**). This corresponds to a gap of roughly **9.47 percentage points**. However, Prodigy–Muon achieves this with substantially less hyperparameter-search compute: the WD-only Prodigy sweep required **114,172** total gradient steps, compared to **402,339** steps for the LR×WD sweep for baseline Muon, i.e., about **3.5× fewer** search steps.

The fact that Armijo–Muon also performs better than Prodigy–Muon over the first 10 epochs reinforces the interpretation that Prodigy–Muon may not be heuristically enforcing the linearization condition as effectively in our setting. In particular, Armijo–Muon matches the tuned Muon baseline within **0.06 percentage points** after 10 epochs (**86.85%** vs. **86.79%**), suggesting that explicitly loss-checked step acceptance can recover much of Muon’s early training performance while removing the need for learning-rate sweeps. This makes Armijo–Muon a valuable ablation for isolating whether Prodigy’s parameter-free distance estimate d_t is the limiting factor.

Observing adaptive distance-parameter (aka learning rate) dynamics for Prodigy–Muon, we find that d_t warms up from around 10^{-6} to roughly 2×10^{-4} and then stabilizes. While d_t is not directly comparable in scale to Muon’s tuned learning rate due to differences in normalization, its relatively cautious plateau is consistent with the hypothesis that Prodigy–Muon is being conservative in the effective step size it selects in Muon’s orthogonalized update space. The staged hyperparameter search for baseline Muon is therefore not a perfect point of comparison for Prodigy–Muon: staged search explicitly reselects fixed LR/WD pairs across segments, whereas Prodigy continuously increases d_t based on its internal loss-based statistics until the linearization signal saturates.

Overall, our results reveal a clear accuracy–compute trade-off for learning-rate-free Muon variants on this benchmark. Armijo–Muon demonstrates that Muon can remain competitive with tuned baselines when step sizes are chosen by explicit, per-batch loss

checks, while Prodigy–Muon significantly reduces search cost by collapsing LR×WD tuning into a WD-only sweep but currently yields lower final accuracy. These findings motivate further alignment with the Prodigy recipe including the recommended γ_k schedule and additional multi-seed, longer-horizon runs to determine whether the present gap is primarily due to implementation choices or a deeper interaction between Prodigy’s distance rule and Muon’s matrix-preconditioned geometry.

8 References

- <https://arxiv.org/pdf/2409.20325> (Story 3 in particular describes how a dynamic learning rate can be implemented. It describes a way to tune learning rates that’s used in the Prodigy optimizer)
- <https://arxiv.org/pdf/2306.06101> (This is the original prodigy optimizer paper, which explains the optimizer and gives benchmarks for the optimizer)
- <https://arxiv.org/pdf/2507.11005> (This is a paper that talks about doing Muon with a second and first order momentum, kind of like Adam. The learning rate is constant, but it’s a similar idea where we modify the steps taken at runtime, but using momentum instead of explicitly changing the learning rate)
- <https://arxiv.org/pdf/2510.05491> (Another paper that has "Neuron-wise" adaptive learning rates. That is, it has an adaptive learning rate for each element of the weight matrices. It does this by basically keeping the L2 norm of the rows of each update to be at about 1).
- <https://arxiv.org/abs/2301.07733> (Proposes a novel algorithm called *D-Adaptation* that automatically sets the learning rate in stochastic optimization, removing the need to manually tune this hyper-parameter, while still achieving the optimal convergence rate for convex Lipschitz problems without extra log-factors.)
- <https://arxiv.org/abs/2509.02046> (A careful LLM pretraining benchmark across scales; key finding: matrix preconditioners (Muon/SOAP) lead, but headline speedups shrink at larger scales, and fair tuning is decisive)
- <https://arxiv.org/pdf/2510.05491> (A parameter-free step-size rule with theory close to optimally tuned GD; conceptually aligned with your gradient-only line-search-style step sizes and a natural alternative to Prodigy/D-Adaptation for ablations.)
- <https://arxiv.org/abs/2409.11321> (Shows that matrix-preconditioned methods can substantially reduce steps and wall-clock versus AdamW, and proposes SOAP as “Adam in the Shampoo eigenbasis,” reducing overhead and stabilizing infrequent eigendecompositions.)
- <https://arxiv.org/pdf/2510.19093> (This paper describes how experiments show that weight decay, and not the scaling in the update rule, of MuP is actually what makes MuP work well, as assumptions that are made in the derivation of MuP fail to hold in practice. For our project, this is important because it suggests that weight decay, along with learning rate, could be important in training.)

9 Appendix

The appendix below summarizes how peer-reviewer feedback on the draft submission was incorporated into the current version of the paper.

Algorithm Naming and Clarity

Reviewer comment: The naming of the proposed algorithm was potentially confusing, particularly the use of “LS-Muon,” which could be conflated with generic line-search variants. **Response:** We renamed the algorithm previously referred to as *LS-Muon* to **Prodigy–Muon** throughout the paper. This change better reflects the algorithm’s core mechanism—the incorporation of Prodigy’s loss-based distance rule into Muon—and avoids ambiguity with classical backtracking line-search methods. The revised naming is used consistently in the text, figures, and Algorithm 1.

Notation Consistency in Update Rules

Reviewer comment: The update rule used inconsistent notation for the adaptive step size, switching between η_t and the Prodigy distance parameter.

Response: We revised the update equations and pseudocode to consistently use the Prodigy distance parameter d_t rather than η_t for Prodigy–Muon. This aligns the notation with the original Prodigy formulation and clarifies that the algorithm is learning a distance-to-step parameter rather than a conventional learning rate. All references in Algorithm 1 and the surrounding discussion have been updated accordingly.

Baseline Muon Hyperparameters

Reviewer comment: For the baseline Muon experiments, several hyperparameters were left at repository defaults and described as “not as important.” The reviewer suggested explicitly stating these values and justifying why they do not materially affect the results.

Response: We clarified in the Methodology section that baseline Muon experiments use the default hyperparameter settings from the public Muon implementation (Jordan Keller’s repository), aside from learning rate and weight decay, which were explicitly swept. We added justification explaining that these defaults are widely used, stable across tasks, and held fixed across all comparisons, ensuring that observed differences are attributable to learning-rate selection mechanisms rather than secondary hyperparameter effects. This improves reproducibility and addresses concerns about hidden tuning.

Gradient-Step Efficiency Comparison

Reviewer comment: A direct comparison of gradient steps versus accuracy for baseline Muon and Prodigy–Muon would strengthen the empirical analysis.

Response: We added explicit reporting of total gradient steps consumed by hyperparameter sweeps for both baseline Muon and Prodigy–Muon. In particular, Section 6 now contrasts the 402,339 gradient steps required for the LR×WD grid search for baseline Muon with the 114,172 steps required for the WD-only sweep for Prodigy–Muon. This comparison makes the compute–accuracy trade-off clearer and directly addresses the reviewer’s request.

Multi-Seed Runs and Error Bars

Reviewer comment: The absence of multi-seed runs and error bars limits the statistical strength of the conclusions.

Response: We explicitly acknowledge this limitation in the Results and Conclusion sections. Due to compute constraints, we were unable to run multiple random seeds or report variance estimates. We now clearly state this as a limitation and outline multi-seed experiments as a priority for future work, particularly to determine whether the observed performance gap for Prodigy–Muon is robust or seed-dependent.